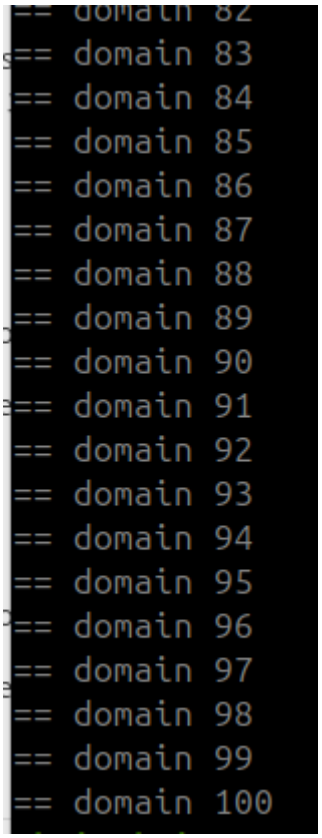
ros2 심화 미들웨어 1일차 보고서

21기 예비단원 최동현

1단계

시도1: 도메인, 토픽, Qos가 무작위로 정해진다고 한다. 일단 Qos는 어떻게 확인하는지 모르겠다. 도메인과 토픽을 확인해 보기로 하였다. 토픽은 무조건 노드가 있어야 있으므로 정확히는 노드를 확인하는 cli를 작성하였다.

```
for i in $(seq 0 101); do
  echo "== domain $i"
  ROS_DOMAIN_ID=$i ros2 node list --no-daemon
done
```



<실행결과>

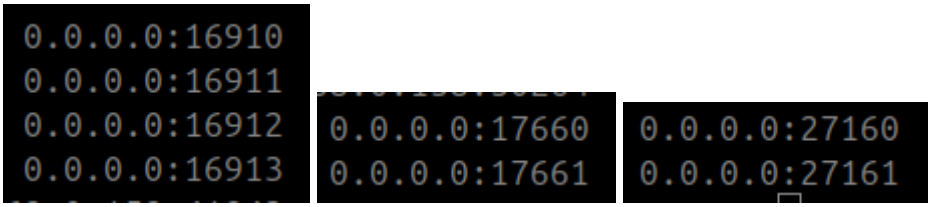
0~101까지 어떠한 노드도 발견할 수 없었다.

시도2: 0~101도메인 중에 정답이 있다면 일단 시도1과 같은 방법으로는 도메인을 찾을 수 없다는 것을 깨닫고 이번에는 포트 번호를 이용하여 도메인을 찾아보기로 하였다.

```
ss -ulpn | grep python3
```

```
UNCONN 0      0            0.0.0.0:16910      0.0.0.0:*    users:(("python3",pid=18729,fd=29))
UNCONN 0      0            0.0.0.0:16911      0.0.0.0:*    users:(("python3",pid=18729,fd=31))
UNCONN 0      0            0.0.0.0:16912      0.0.0.0:*    users:(("python3",pid=18727,fd=29))
UNCONN 0      0            0.0.0.0:16913      0.0.0.0:*    users:(("python3",pid=18727,fd=31))
UNCONN 0      0      192.168.0.158:41843      0.0.0.0:*    users:(("python3",pid=18723,fd=33))
UNCONN 0      0      192.168.0.158:50264      0.0.0.0:*    users:(("python3",pid=18727,fd=33))
UNCONN 0      0            0.0.0.0:17660      0.0.0.0:*    users:(("python3",pid=18725,fd=29))
UNCONN 0      0            0.0.0.0:17661      0.0.0.0:*    users:(("python3",pid=18725,fd=31))
UNCONN 0      0            0.0.0.0:50676      0.0.0.0:*    users:(("python3",pid=18729,fd=32))
UNCONN 0      0            0.0.0.0:50763      0.0.0.0:*    users:(("python3",pid=18727,fd=32))
UNCONN 0      0            0.0.0.0:54969      0.0.0.0:*    users:(("python3",pid=18725,fd=32))
UNCONN 0      0            0.0.0.0:59469      0.0.0.0:*    users:(("python3",pid=18723,fd=32))
UNCONN 0      0      192.168.0.158:51474      0.0.0.0:*    users:(("python3",pid=18729,fd=33))
UNCONN 0      0      192.168.0.158:39298      0.0.0.0:*    users:(("python3",pid=18725,fd=33))
UNCONN 0      0            0.0.0.0:27160      0.0.0.0:*    users:(("python3",pid=18723,fd=29))
UNCONN 0      0            0.0.0.0:27161      0.0.0.0:*    users:(("python3",pid=18723,fd=31))
```

<포트 7400이하를 제거한 실행결과>
192.168~는 os가 임의로 준 포트여서 제외하면



이렇게가 유력한 것으로 보인다.



이식을 이용하여 계산을 하면 (16910-7400)/250=38, (17660-7400)/250=41, (27160-7400)/250=79 이렇게 3개의 도메인이 나온다.

시도3: 이제 38, 41, 79에 대한 도메인 재검사를 해봐야 한다. cli는

```
for d in 38 41 79; do
  echo "== domain $d"
  ROS_DOMAIN_ID=$d ros2 topic list --no-daemon
done
```

이렇게 사용하였다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ for d in 38 41 79; do
  echo "== domain $d"
  ROS_DOMAIN_ID=$d ros2 topic list --no-daemon
done
== domain 38
/parameter_events
/rosout
== domain 41
/parameter_events
/rosout
== domain 79
/parameter_events
/rosout
```

<실행결과>
결국 이 도메인에서도 노드가 뜨지 않았다.

시도4: 포트와 도메인중에 누군가는 거짓말을 하고 있는 것이었다. 근데 포트를 알아내는 cli는 뭔가를 건드릴 부분이 없다면 도메인 cli는 ros2 topic list --no-daemon이 부분이 내 예상과 다른 결과를 만들어 낼 수도 있는 가능성이 있다고 생각하였다.

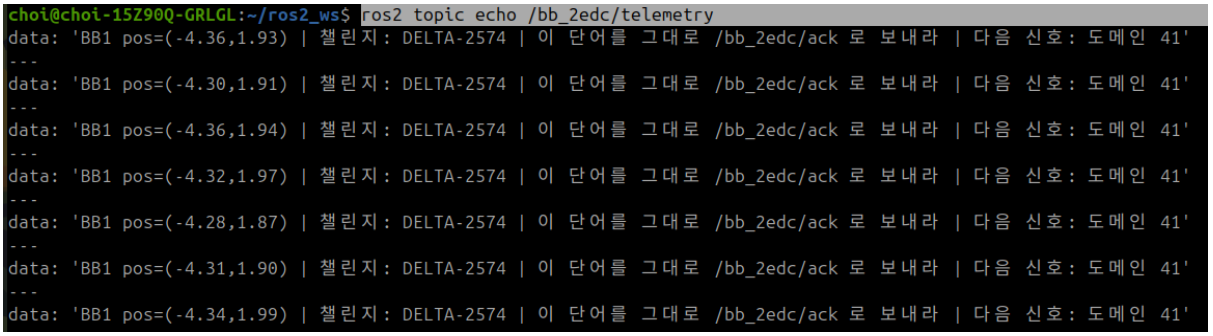
그래서 --spin-time 5이 부분을 추가하여서 cli를 다시 검사하였다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ROS_DOMAIN_ID=38 ros2 topic list --no-daemon --spin-time 5
/bb_2edc/cmd
/parameter_events
/rosout
```

결국 발견 대기 시간이 너무 짧은 것이 이유였다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ for d in 38 41 79; do
  echo "== domain $d"
  ROS_DOMAIN_ID=$d ros2 topic list --no-daemon --spin-time 3
done
== domain 38
/bb_2edc/cmd
/parameter_events
/rosout
== domain 41
/bb_2edc/answer
/bb_2edc/config
/bb_2edc/stream
/parameter_events
/rosout
== domain 79
/bb_2edc/ack
/bb_2edc/telemetry
/parameter_events
/rosout
```

<실행결과>
추가적으로 41,79 도메인도 실행해보았다.
1단계가 telemetry에 대한 것이므로



telemetry토픽에서 나오는 메세지를 읽어 보았다. DELTA-2574를 ack으로 보내라는 메세지가 출력되었다.

```
^Cchoi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/ack
Type: std_msgs/msg/String

Publisher count: 0

Subscription count: 1

Node name: bb_s1
Node namespace: /
Topic type: std_msgs/msg/String
```

ack이 string형식인 것을 확인하고

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic pub --once -w 1 /bb_2edc/ack std_msgs/msg/String "{data: 'DELTA-2574'}"
Waiting for at least 1 matching subscription(s)...
publisher: beginning loop
publishing #1: std_msgs.msg.String(data='DELTA-2574')

choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 run robit_blackbox status --name choi
이름 : choi
  1단계 정찰                통과   2026-11-05T22:21:56+09:00
  2단계 QoS                -
  3단계 Cyclone 설정         -
  4단계 고장 진단 (도전)        -
  5단계 자유 탐구        보고서로 제출
```

ack으로 DELTA_2574를 보내서 1단계 정찰을 통과하였다.

2단계

Qos와 config, stream에 대한 정보를 알아볼려고 한다. 아까 사용하였던 cli중에 Qos의 정보를 나타내주는 cli가 있었어서 재사용할려고 한다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ export ROS_DOMAIN_ID=41
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ echo $ROS_DOMAIN_ID
41
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/config
Unknown topic '/bb_2edc/config'
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/config
Type: std_msgs/msg/String

Publisher count: 1

Node name: bb_s2
Node namespace: /
Topic type: std_msgs/msg/String
Topic type hash: RIHS01_df668c740482bbd48fb39d76a70dfd4bd59db1288021743503259e948f6b1a18
Endpoint type: PUBLISHER
GID: 01.0f.94.37.25.49.8a.a4.00.00.00.00.00.00.13.03
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: TRANSIENT_LOCAL
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 0
```

<config info>

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/stream
Type: std_msgs/msg/String

Publisher count: 1

Node name: bb_s2
Node namespace: /
Topic type: std_msgs/msg/String
Topic type hash: RIHS01_df668c740482bbd48fb39d76a70dfd4bd59db1288021743503259e948f6b1a18
Endpoint type: PUBLISHER
GID: 01.0f.94.37.25.49.8a.a4.00.00.00.00.00.00.14.03
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 0
```

<stream info>

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/answer
Type: std_msgs/msg/String

Publisher count: 0

Subscription count: 1

Node name: bb_s2
Node namespace: /
Topic type: std_msgs/msg/String
Topic type hash: RIHS01_df668c740482bbd48fb39d76a70dfd4bd59db1288021743503259e948f6b1a18
Endpoint type: SUBSCRIPTION
GID: 01.0f.94.37.25.49.8a.a4.00.00.00.00.00.00.15.04
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

<answer info>

도메인을 바꾸어주고 topic info를 확인하였다 (추가적으로 answer까지).

	Reliability	Durability
config	RELIABE	TRANSIENT_LOCAL
stream	BEST_EFFORT	VOLATILE
answer	RELIABE	VOLATILE

config를 sub할려면 reliable, transient local

stream을 sub할려면 best effort, volatile
answer에 pub을 할려면 reliable, transient local로 설정해야 한다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic echo /bb_2edc/config std_msgs/msg/S
tring --qos-reliability reliable --qos-durability transient_local --full-length
data: 'BB2-A word=8fe2 | 스트림은 /bb_2edc/stream | 각 메시지의 word= 뒤 값 두
개를 A-B 순서로 "-" 로 이어 /bb_2edc/answer 로 보내라 (구독 노드가 켜져 있는 동
안) | 다음 신호: 도메인 38의 /bb_2edc/cmd'
---
^Cchoi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic echo /bb_2edc/stream std_msgs/msg
/String --qos-reliability best_effort --full-length
data: BB2-B word=0404 n=546
---
data: BB2-B word=0404 n=547
---
data: BB2-B word=0404 n=548
---
```

<config와 stream이 보내는 메세지 출력 결과>
word뒤에 나오는 값을 -기호로 이어서 answer로 보내라는 것이다.

```
^Cchoi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 run bb_solver stage2_node
[INFO] [1793948829.614962767] [stage2_node]: stage2_node 시작. config/stream 기
다리는 중...
[INFO] [1793948831.356100508] [stage2_node]: stream 수신: B=0404
[INFO] [1793948831.420109808] [stage2_node]: config 수신: A=8fe2
[INFO] [1793948831.615353486] [stage2_node]: answer 발행: 8fe2-0404
[INFO] [1793948832.615348629] [stage2_node]: answer 발행: 8fe2-0404
[INFO] [1793948833.615400107] [stage2_node]: answer 발행: 8fe2-0404
[INFO] [1793948834.615663826] [stage2_node]: answer 발행: 8fe2-0404
```

<answer로 수신한 결과>

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 run robit_blackbox status --name choi
이름: choi
  1단계 정찰                통과   2026-11-05T22:21:56+09:00
  2단계 QoS               통과   2026-11-06T01:05:51+09:00
  3단계 Cyclone 설정        통과   2026-11-06T15:21:21+09:00
  4단계 고장 진단 (도전)        -
  5단계 자유 탐구        보고서로 제출
```

<2단계가 통과한 status 결과>
3단계까지 끝내고 찍은 결과여서 3단계 통과도 함께 출력되었다.

3단계

네트워크를 세부적으로 설정해서 통신하는 cyclone dds를 사용한다.

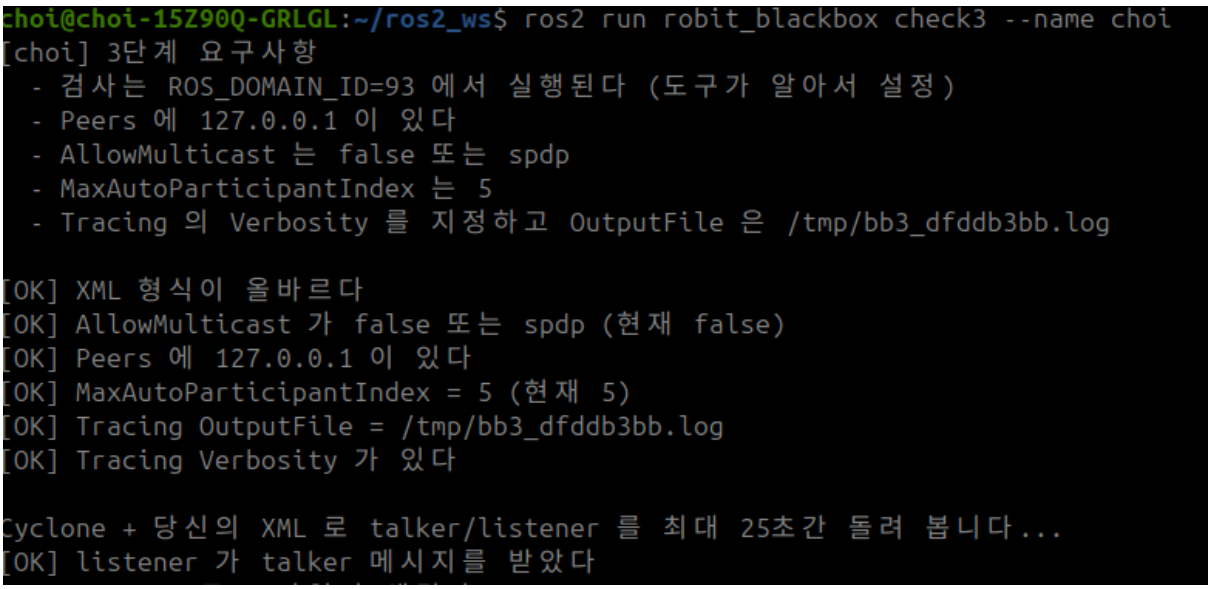
```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 run robit_blackbox check3 --name choi
[choi] 3단계 요구사항
  - 검사는 ROS_DOMAIN_ID=93 에서 실행된다 (도구가 알아서 설정)
  - Peers 에 127.0.0.1 이 있다
  - AllowMulticast 는 false 또는 spdp
  - MaxAutoParticipantIndex 는 5
  - Tracing 의 Verbosity 를 지정하고 OutputFile 은 /tmp/bb3_dfddb3bb.log

[NG] 설정 파일이 없습니다: /home/choi/cyclonedds_bb.xml
[ros2run]: Process exited with failure 1
```

<cyclone의 요구사항의 출력 결과>
peer는 127.0.0.1로 설정하기. allowMulticast는 false로 설정하기.
maxautoparticipantindex는 5로 설정하기. tracing은 verbosity+outputfile로 하기
도메인은 도구가 정하기나깐 xml에서 고정하지 않았다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ cat > ~/cyclonedds_bb.xml << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<CycloneDDS xmlns="https://cdds.io/config">
  <Domain Id="any">
    <General>
      <AllowMulticast>false</AllowMulticast>
    </General>
    <Discovery>
      <ParticipantIndex>auto</ParticipantIndex>
      <MaxAutoParticipantIndex>5</MaxAutoParticipantIndex>
      <Peers>
        <Peer address="127.0.0.1"/>
      </Peers>
    </Discovery>
    <Tracing>
      <Verbosity>config</Verbosity>
      <OutputFile>/tmp/bb3_dfddb3bb.log</OutputFile>
    </Tracing>
  </Domain>
</CycloneDDS>
```

<xml파일을 제작한 결과>



<출력결과>
3단계를 통과하였다.

4단계

bb_2edc/cmd 토픽의 잘못된 talker와 listener의 문제점을 발견하고 고쳐주는 것이다.

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/cmd
Type: ['std_msgs/msg/Int32', 'std_msgs/msg/String']

Publisher count: 1

Node name: bb_talker4
Node namespace: /
Topic type: std_msgs/msg/String
Topic type hash: RIHS01_df668c740482bbd48fb39d76a70dfd4bd59db1288021743503259e948f6b1a18
Endpoint type: PUBLISHER
GID: 01.0f.94.37.51.25.15.69.00.00.00.00.00.00.13.03
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Subscription count: 1

Node name: bb_listener4
Node namespace: /
Topic type: std_msgs/msg/Int32
Topic type hash: RIHS01_b6578ded3c58c626cfe8d1a6fb6e04f706f97e9f03d2727c9ff4e74b1cef0deb
Endpoint type: SUBSCRIPTION
GID: 01.0f.94.37.53.25.bd.9d.00.00.00.00.00.00.13.04
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

<topic상태 출력결과>

다행히 talker와 listener가 잘 살아있는 상태였다. 근데 문제점은 topic type과 Reliability였다 .

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ for d in 38 41 79; do
  echo "== domain $d"
  ROS_DOMAIN_ID=$d ros2 topic list --no-daemon --spin-time 3
done
== domain 38
/bb_2edc/cmd
/parameter_events
/rosout
== domain 41
/bb_2edc/answer
/bb_2edc/config
/bb_2edc/stream
/parameter_events
/rosout
== domain 79
/bb_2edc/ack
/bb_2edc/telemetry
/parameter_events
/rosout
```

	domain	topic	topic type	reliability
talker	38	bb_2edc/cmd	string	best_effort
listener	38	bb_2edc/cmd	int_32	reliable

domian과 topic은 모든 도메인을 1단계에서 찾아보았때 추가로 나오지 않고 topic info에서 talker, listener 둘다 나왔으므로 증명되었다.
listener를 stirng과 best_effort로 바꾸면 된다.

```
1  # 로봇 listener 의 설정. 이 파일을 고치면 listener 가 자동으로 다시 시작된다.
2  # 값 중 일부가 틀려 있다. 로봇이 보내는 신호를 조사해서 고쳐라.
3  LISTENER_DOMAIN=38
4  LISTENER_TOPIC=/bb_2edc/cmd
5  LISTENER_TYPE=String
6  LISTENER_RELIABILITY=best_effort
7
```

<listener를 바꾼 출력결과>

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 topic info -v /bb_2edc/cmd --no-daemon
Type: std_msgs/msg/String

Publisher count: 0

Subscription count: 1

Node name: bb_listener4
Node namespace: /
Topic type: std_msgs/msg/String
Topic type hash: RIHS01_df668c740482bbd48fb39d76a70dfd4bd59db1288021743503259e948f6b1a18
Endpoint type: SUBSCRIPTION
GID: 01.0f.94.37.cf.2e.4e.07.00.00.00.00.00.00.13.04
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

<talker가 할일을 끝내고 사라진 출력결과>

```
choi@choi-15Z90Q-GRLGL:~/ros2_ws$ ros2 run robit_blackbox robot --name choi
블랙박스 로봇이 켜졌습니다. (이름: choi)
  진행 확인      : ros2 run robit_blackbox status --name "choi"
  listener 설정 : /home/choi/.ros/robit_blackbox/dfddb3bb/listener.env  (4단계)
이 터미널은 그대로 두세요. 종료는 Ctrl+C. 같은 이름으로 다시 켜면 진행이 이어집
니다.
listener.env 변경을 감지해 listener 를 다시 시작했습니다.

★ 4단계 통과! 기록되었습니다.
```

```
^Cchoi@choi-15Z90Q-GRLGL:~/ros2_wsros2 run robit_blackbox status --name choi
이름: choi
  1단계 정찰                  통과   2026-11-05T22:21:56+09:00
  2단계 QoS                 통과   2026-11-06T01:05:51+09:00
  3단계 Cyclone 설정          통과   2026-11-06T15:21:21+09:00
  4단계 고장 진단 (도전)          통과   2026-11-06T17:02:49+09:00
  5단계 자유 탐구          보고서로 제출
choi@choi-15Z90Q-GRLGL:~/ros2_ws$
```

<4단계 통과 출력 결과>

아직이해하지 못한 것

cyclone dds는 통신을 세부적으로 설정하기 위한 것이라고 이해하였습니다. 또한 세부적인 설정을 하여 필요한 곳에서 값을 주고 받는데 효과적이라고도 생각합니다. 그런데 cyclone에 allowmulticast에 true 항목이 있는 이유는 무엇이며, 실제로 사용을 하기도 하는지 궁금합니다. 사용을 한다면 어떤 상황에서 사용하는지도 궁금합니다.